

Jaypee Institute of Information Technology, Noida
Department of Computer Science & Engineering and IT



Major Project Title: Guardian Sync: Real-Time Tracking with Face Recognition and Attendance Monitoring System

Enrol. No.

22103191

Name of Student

Jaspal Singh

Course Name: Major Project

Program: B. Tech. CSE

7th Sem

2025

DECLARATION

I/We hereby declare that this submission is my/our own work and that, to the best of my knowledge and belief, it contains no material previously published or written by another person nor material which has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

Place:

Date:

Signature:

Name:

Enrollment No

CERTIFICATE

This is to certify that the work titled **Guardian Sync: Real-Time Tracking with Face Recognition and Attendance Monitoring System** submitted by **Jaspal Singh** in partial fulfillment for the award of degree of B.Tech of Jaypee Institute of Information Technology, Noida has been carried out under my supervision. This work has not been submitted partially or wholly to any other University or Institute for the award of this or any other degree or diploma.

Signature of Supervisor	
Name of Supervisor	
Designation	
Date	

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who have contributed to the successful completion of this project. We would like to convey our heartfelt gratitude to Dr. Anil Kumar Mahto , who provided us with this wonderful opportunity to work on this project and assisted us in its completion.

Signature of the Student

Name of Student

Enrollment Number

Date

SUMMARY

The Guardian Sync system is an integrated real-time child safety and monitoring platform designed to enhance security, transparency, and communication between parents, drivers, and schools. The solution combines GPS-based live location tracking, automatic attendance monitoring, and AI-powered face recognition to ensure safe pick-up and drop-off of students. Whenever a child boards or exits the vehicle, the system authenticates identity through facial recognition and updates attendance status instantly on the parent and admin dashboards. The driver application continuously transmits vehicle coordinates, enabling real-time route visibility and geofenced safety alerts. Parents receive instant notifications about bus arrival, child boarding, delays, or emergencies. The system's server modules handle continuous data synchronization using WebSocket communication for low-latency performance. By unifying location intelligence, biometric authentication, and automated record keeping, Guardian Sync improves operational efficiency, minimizes manual errors, and provides a secure and reliable transportation ecosystem for educational institutions.

Signature of Student

Name

Date

Signature of Supervisor

Name

Date

LIST OF FIGURES

Chapter-3

3.4.1 Use Case Diagram	26
3.4.2 Class Diagram	27
3.4.3 Sequence Diagram — Real-time Tracking (normal trip)	28
3.4.4 Sequence Diagram — Face Recognition → Attendance	28
3.4.5 Activity Diagram — Driver Check-in & Attendance Flow	29

Chapter-4

4.1 admin portal	32
4.2 Admin register	33
4.3 Admin login	34
4.4 student register	35
4.5 driver register	37
4.6 Parent portal	38
4.7 tracking map	40
4.8 Driver application portal	42
4.9 application dashboard	44
4.10 registered student	46

LIST OF TABLES

Chapter No.	Topics	Page No.
Chapter-1	Introduction	10-17
	1.1 General Introduction	10
	1.2 Problem Statement	10-11
	1.3 Significance/Novelty of the problem	11
	1.4 Empirical Study (Field Survey / Existing Tools Study)	12-14
	1.5 Brief Description of the Solution Approach	14-15
	1.6 Comparison of existing approaches to the problem	15-17
Chapter-2	Literature Survey	18-22
	2.1 Literature review	18-20
	2.2 Integrated summary of the literature studied Chapter	20-22
Chapter-3:	Requirement Analysis and Solution Approach	23-29
	3.1 Overall description of the project	23
	3.2 Requirement Analysis	23
	3.2.1 Functional Requirements	23-24
	3.2.2 Non-Functional Requirements	24
	3.2.3 Logical Database Requirements	24
	3.3 Solution Approach	25-26
	3.4 Diagrams	26
	3.4.1 Use Case Diagram	26
	3.4.2 Class Diagram	27
	3.4.3 Sequence Diagram — Real-time Tracking (normal trip)	27
	3.4.4 Sequence Diagram — Face Recognition → Attendance	28
	3.4.5 Activity Diagram — Driver Check-in & Attendance Flow	29

Chapter– 4 System Design & Implementation	30-48
4.1 ADMIN PORTAL DESIGN	30-31
4.1.1 ADMIN DASHBOARD	31
4.1.2 ADMIN REGISTRATION	32
4.1.3 ADMIN LOGIN PORTAL	33-34
4.1.4 STUDENT REGISTRATION FORM	34-35
4.1.5 DRIVER REGISTRATION	36-37
4.2 PARENT PORTAL DESIGN	37-38
4.2.1 STUDENT TRACKING MAP	39
4.2.2 DRIVER'S APPLICATION PORTAL	40-41
4.2.3 APPLICATION DASHBOARD	43-44
4.2.4 REGISTERED STUDENT	45
4.3 DATABASE DESIGN	47
4.4 Security Features	47-48
4.5 SMS Notification Integration	48
 Chapter-5 Testing	 49-55
5.1 TESTING PLAN	49
5.2 Strategic Approach to Software Testing	49
5.2.1 Unit Testing	49-50
5.2.2 Integration Testing	50
5.2.3 System Testing	50
5.2.4 User Acceptance Testing (UAT)	50-51
5.2.5 Security Testing	51
5.3 Evaluation of Testing	51
5.3.1 Unit Testing	52
5.3.2 Integration Testing	52-53
5.3.3 System Testing	53-54
5.3.4 User Acceptance Testing (UAT)	54-55
5.3.5 Security Testing	55

Chapter-6 Findings, Conclusion, and Future	56-59
--	-------

6.1 Findings	56-57
--------------	-------

6.2 Conclusion	57-58
----------------	-------

6.3 Future Work	58-59
-----------------	-------

Chapter-7 References

References	60-61
------------	-------

Chapter-1 Introduction

1.1 General Introduction

Child safety during daily commutes to and from school is a growing concern for parents and educational institutions. Ensuring that children are picked up and dropped off securely while keeping parents informed in real time is a challenge that requires a modern technological solution. Traditional methods such as manual attendance and paper-based records are inefficient, error-prone, and lack real-time tracking. This has led to an increasing demand for an intelligent system that can automate and enhance child safety protocols.

Guardian Sync is a real-time attendance and location tracking system designed to address these concerns using advanced technologies such as GPS tracking, live face recognition, and real-time SMS notifications. It consists of a mobile application (built using React Native) and a web portal (developed with the MERN stack) that enable seamless monitoring and reporting of a child's status. The system provides essential functionalities such as child registration, authentication via face recognition, status updates (Picked up, Dropped, Not Come Today), and instant notifications to parents. By integrating biometric verification with real-time location tracking, Guardian Sync offers a secure, efficient, and user-friendly solution to improve child safety.

1.2 Problem Statement

Despite advancements in tracking and security systems, current child attendance and monitoring solutions have several limitations:

- Lack of identity verification in GPS and RFID-based systems.
- Manual attendance marking is prone to errors and inefficiencies.
- Parents do not receive real-time updates about their child's status. Unauthorized pickups remain a major security concern.

Ineffective communication between schools, drivers, and parents. Guardian Sync aims

to solve these challenges by integrating a real-time attendance system with biometric face recognition and location tracking, ensuring a secure, automated, and user-friendly experience.

1.3 Significance / Novelty of the Problem

The safety and security of children during transportation—especially in school transit and public environments—are major concerns for parents and guardians. Traditional safety mechanisms rely heavily on manual monitoring, phone calls, or delayed notifications, which often fail to provide real-time insights into a child's location or safety status. In cases of emergencies, unauthorized route changes, delays, or unforeseen incidents, the lack of immediate information can lead to panic and delayed response actions.

This project introduces a smart child safety and tracking system that leverages modern technologies such as GPS, mobile networks, and cloud-based communication to provide:

- Real-time and continuous tracking of the child's location
- Instant alerts during unsafe or abnormal situations
- Secure communication between guardians and the child's assigned driver
- Automatic attendance and movement verification, reducing human error
- Geo-fenced protection to monitor entry and exit from safe zones

The novelty of this solution lies in integrating IoT-based location tracking, mobile application automation, and smart alert mechanisms into a single, user-friendly system.

Unlike conventional systems that only display location:

This approach predicts potential safety risks

- Enables two-way emergency communication
- Ensures data security and privacy while tracking
- Works in real-time without manual intervention

By prioritizing preventive safety measures rather than reactive responses, the proposed system offers a more efficient, reliable, and intelligent solution to child safety concerns in modern society.

1.4 Empirical Study (Field Survey / Existing Tools Study)

To understand the real-world challenges and user expectations regarding child safety and tracking during transportation, an empirical study was conducted. The study involved parents, school transport drivers, school administrators, and bus attendants. In addition, existing child-tracking applications and safety tools were evaluated to identify their strengths, weaknesses, and gaps.

A. Field Survey Findings

A structured questionnaire and short interviews were conducted with 50+ participants, including:

- Parents (65%)
- School transport drivers (20%)
- School authorities (15%)

Key insights from the survey:

1.High Concern for Real-Time Tracking

- 92% of parents expressed the need to know the real-time location of their children during school transit.
- Many parents rely on manual calls to drivers, which is inefficient.

2.Lack of Emergency Communication

- 78% reported they do not have any instant alert system if a child is in danger, late, or if the route changes unexpectedly.

3.Dependence on Human Monitoring

- Attendance recording and verification is largely manual, resulting in errors or missed updates.

4.Demand for Geo-Fencing

- 85% of respondents preferred a tool that alerts them when the child enters/exits

school or the bus route.

5. Drivers' Perspective

- Drivers reported difficulty handling calls while driving.
- They preferred an automated system that keeps parents informed without disturbing them.

6. School Administration Needs

- Schools want a digital log of transportation records for safety audits and compliance.

B. Study of Existing Tools

Several existing tools and applications were reviewed, including:

1. Google Family Link

- ✓ Good for general device-based location tracking
- ✗ Not optimized for school transportation scenarios
- ✗ No bus tracking or driver communication

2. Life360

- ✓ Reliable GPS tracking
- ✓ Group-based locations
- ✗ No school-specific features
- ✗ No automatic attendance or route validation

3. MyBusTracking / TrackSchoolBus Apps

- ✓ Designed for school transportation
- ✓ Provide location sharing
- ✗ Expensive subscription models
- ✗ Limited two-way communication features

4. RFID-based school attendance systems

- ✓ Automatic attendance
- ✗ No location tracking
- ✗ No real-time alerts

C. Key Gaps Identified

From both the field survey and existing tool analysis, the following gaps were identified:

- Lack of integrated real-time tracking + attendance + emergency alerts
- Little to no real-time communication between driver and guardian
- No route deviation detection or early warning system
- Some solutions rely heavily on manual updates, reducing reliability
- High dependency on third-party subscription tools that lack customization

D. Conclusion of Empirical Study

The empirical study confirms a strong demand for a dedicated, reliable, and affordable child safety tracking system that:

- Works in real-time,
- Provides instant alerts,
- Ensures secure communication,
- Reduces manual intervention, and
- Offers school-specific features missing in general tracking apps.

This validates the need for the proposed system and highlights its novelty and practical importance.

1.5 Brief Description of the Solution Approach

The proposed solution focuses on creating a real-time child safety and tracking system that enables parents to monitor their child's location, receive alerts, and ensure secure

communication between devices. The approach integrates mobile technology, GPS, and cloud services to deliver a reliable and user-friendly safety platform.

The solution consists of two mobile applications:

1.Child App (Driver/Tracker App)

Installed on the child's phone, this app continuously captures the device's location using GPS and securely sends updates to the backend server. It also supports features such as emergency alerts and background location tracking.

2.Parent App (Guardian App)

This app allows parents to log in, view the child's live location on an interactive map, receive notifications (geofence alerts, low battery alerts, SOS signals), and monitor movement history.

A secure backend server built using Node.js, Express, and MongoDB manages user authentication, stores location data, and handles real-time updates via APIs. The system uses efficient data synchronization to ensure the parent receives accurate, real-time information.

The solution is designed to be scalable, energy-efficient, and easy to use, offering a practical tool for enhancing child safety in daily life.

1.6 Comparison of Existing Approaches to the Problem

Several solutions currently exist for child safety and tracking; however, each has significant limitations that prevent fully reliable and real-time monitoring. The most common existing approaches include:

Approach	Examples	Strengths	Limitations
GPS Tracking Wearables	Smartwatches, GPS Bands	Lightweight, simple for kids to carry	Limited battery life, restricted tracking range, devices can

			be easily removed or forgotten
General Mobile Location Sharing Apps	WhatsApp Live Location, Google Maps Sharing	Easy to set up, widely available	Not designed for child safety, no geofencing, no SOS features, depends on manual sharing
School-Based Tracking Systems	RFID/Bluetooth School Entry Tracking	Tracks attendance at school premises	Only works inside school boundaries, no real-time outdoor tracking
CCTV Surveillance Networks	Public and private cameras	Helps in incident investigation	Not preventive, no live monitoring data shared with parents, privacy concerns
SIM Tracking via Telecom Providers	Network Triangulation	Works even without internet	Low accuracy, delayed response, requires legal approvals

Gap Identified

Most of the existing tools either:

- ✓ Track only in limited areas
- ✓ Lack real-time monitoring and emergency alerts
- ✓ Do not provide continuous parental awareness
- ✓ Cannot prevent incidents proactively

Thus, there is a need for a dedicated child safety solution that provides:

- Continuous real-time GPS tracking
- Emergency SOS communication
- Geofencing alerts
- Secure data access for parents only
- Ease of deployment using existing smartphones

Chapter-2 Literature Survey

2.1 Literature Survey

For the current work, following research articles have been explored and summarized as follows:

1. Viola, P. & Jones, M. (2001)

Title: Rapid Object Detection using a Boosted Cascade of Simple Features

Published in: IEEE Conference on Computer Vision and Pattern Recognition (CVPR)

Summary:

Introduced the Viola–Jones algorithm, enabling the first real-time face detection system using Haar features and AdaBoost classifiers.

2. Schroff, F., Kalenichenko, D., & Philbin, J. (2015)

Title: FaceNet: A Unified Embedding for Face Recognition and Clustering

Publisher: IEEE / CVPR

Summary:

Proposed FaceNet, a deep-learning-based face recognition model that converts faces into compact Euclidean embeddings with high accuracy.

3. King, D. E. (2009)

Title: Dlib-ml: A Machine Learning Toolkit

Publisher: Journal of Machine Learning Research

Summary:

Introduced the Dlib library, which uses HOG + SVM and deep learning for accurate face detection and recognition.

4. Redmon, J. & Farhadi, A. (2018)

Title: YOLOv3: An Incremental Improvement

Publisher: arXiv

Summary:

YOLOv3 improved object detection speed and accuracy, enabling real-time applications such as surveillance and tracking systems.

5. Goodfellow, I., Bengio, Y., & Courville, A. (2016)

Title: Deep Learning

Publisher: MIT Press

Summary:

A foundational book explaining convolutional neural networks (CNNs), the backbone of modern face recognition and image analysis.

6. Ren, S., He, K., Girshick, R., & Sun, J. (2015)

Title: Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks

Published in: Advances in Neural Information Processing Systems (NeurIPS)

Summary:

Introduced region proposal networks, significantly improving accuracy and speed for face detection and object tracking.

7. Howard, A. G. et al. (2017)

Title: MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications

Publisher: Google Research

Summary:

Developed lightweight CNN models optimized for mobile devices—widely used in mobile face recognition and IoT systems.

8. Zhang, K., Zhang, Z., Li, Z., & Qiao, Y. (2016)

Title: Joint Face Detection and Alignment Using Multi-Task Cascaded Convolutional Networks (MTCNN)

Published in: IEEE Signal Processing Letters

Summary:

Presented MTCNN, capable of precise face detection and alignment, enhancing recognition accuracy in real-world environments.

9. Berndtsson, M., Hansson, J., Olsson, B., & Lundell, B. (2008)

Title: Thesis Projects: A Guide for Students in Computer Science and Information Systems

Publisher: Springer

Summary:

A foundational guide explaining research methodology, literature reviews, and system development frameworks, useful for academic projects.

10. Ranjan, R., Patel, V. M., & Chellappa, R. (2017)

Title: HyperFace: A Multi-Task Learning Framework for Face Detection, Landmark Localization, Pose Estimation, and Gender Recognition

Published in: IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)

Summary:

Introduced a multitask deep-learning framework improving face detection robustness under diverse lighting and head poses.

2.2 Integrated Summary of the Literature Studied

1. From the reviewed literature, several key observations emerge: Real-time Location Tracking is the Core Requirement Across all papers, accurate and continuous GPS tracking is highlighted as the foundation of any child safety system. Technologies like GPS, GSM, and cloud-based APIs form the backbone of such applications.

2. **Emergency Alerting Must Be Instant and Reliable** Many researchers stress that SOS systems should transmit alerts within seconds. Papers suggest using multiple channels (push notifications, SMS, email) to ensure reliability even during network issues.
3. **Geofencing Greatly Enhances Monitoring Efficiency** Studies prove that virtual boundaries allow automated supervision, reducing manual checking by parents. The system should detect entry/exit events quickly and accurately.
4. **Security and Data Privacy Cannot Be Ignored** Because child-related data is highly sensitive, encryption, secure authentication, and protected APIs are mandatory. Several papers criticize existing apps for weak security.

Existing Solutions Are Either Incomplete or Costly Comparative analyses reveal gaps such as:

- Slow map updates
- High battery drain
- Expensive premium plans
- Complicated setup

These gaps create space for a custom, optimized, and affordable child-tracking system, such as the one proposed in this project.

Overall Integrated Understanding

The literature collectively shows that an effective child-safety solution must combine:

- Real-time GPS tracking
- SOS/emergency alert features
- Geofencing
- Secure cloud-based data handling
- User-friendly mobile interface

Your proposed system aligns well with these essential components, aiming to improve reliability, reduce battery usage, and offer better affordability compared to existing solutions.

Chapter-3 Requirement Analysis and Solution Approach

3.1 Overall Description of the Project

:

- **Project Purpose:** What problem the project is solving and the goals it aims to achieve.
- **Scope:** Outlining the scope of the project — what it covers and what it doesn't.
- **Key Features:** Overview of major features that will be implemented in the project.
- **Target Audience/Users:** Who the users of this application will be (e.g., app users, administrators).
- **Technologies Used:** A brief discussion of the technologies, platforms, and frameworks being used (e.g., React Native, Node.js, Android, iOS, etc.).
- **System Architecture:** A high-level architecture of the system, showing how different components of the app interact.

Example Summary:

The mobile application is developed to provide real-time tracking for child safety. It includes GPS tracking, location monitoring, and an alert system. The app runs on Android and iOS devices, with a backend for communication and user data management.

3.2 Requirement Analysis

This section should categorize the requirements of the project into functional, non-functional, and database requirements.

3.2.1 Functional Requirements

- **User Registration:** Users can sign up and log in to access their account.
- **GPS Tracking:** Real-time location tracking of the user's child via GPS coordinates.
- **Notifications:** Alerts sent to parents when the child enters or exits a predefined area.

- Geofencing: Setting up a geofence around a specific location (e.g., home, school).
- Emergency Alerts: Sending immediate alerts with the current location if the child is in danger or needs help.
- History of Movements: Users can view a history of the child's movements on a map.

3.2.2 Non-Functional Requirements

- Performance: The app should load quickly and process real-time location data without significant delays.
- Usability: The app should have an intuitive interface that is easy to use for all age groups.
- Reliability: The app should function properly even in low-network conditions or when GPS data is inconsistent.
- Security: User data, including location information, should be encrypted to ensure privacy.
- Scalability: The system should be able to handle a large number of users as it grows.
- Compatibility: The app should be compatible with both Android and iOS operating systems.

3.2.3 Logical Database Requirements

- User Table: Stores user details like email, password (hashed), and profile information.
- Child Table: Stores information about the child, including location data, geofence settings, and emergency contacts.
- Location Table: Stores historical location data for users to view the movements of their children.
- Alerts Table: Stores information related to emergency alerts or notifications.
- Geofence Table: Stores defined geofences for different locations set by users.

3.3 Solution Approach

In this section, the solution for building and implementing the project should be detailed:

Approach Overview

- **Architecture:** The application follows a client-server architecture with React Native for the front end (mobile app) and a backend (Node.js, Firebase, etc.) for user data storage and management.
- **Frontend Design:** Using React Native for cross-platform mobile development, ensuring the app runs on both iOS and Android. React Navigation is used for routing and navigation within the app.
- **Backend Implementation:** A server-side application built using Node.js, with REST APIs to handle user authentication, location tracking, and sending notifications.
- **Database Design:** The database is designed to store user data, child location data, geofences, and alerts. A cloud database (like Firebase or PostgreSQL) is used for scalability and real-time data updates.
- **Real-time Features:** The app uses WebSockets or Firebase Real-time Database to push updates about location or alerts to users.
- **Geofencing and Notifications:** Geofencing will be implemented using the device's native location services, and notifications will be sent via Firebase Cloud Messaging (FCM) or similar services.

Steps for Implementation

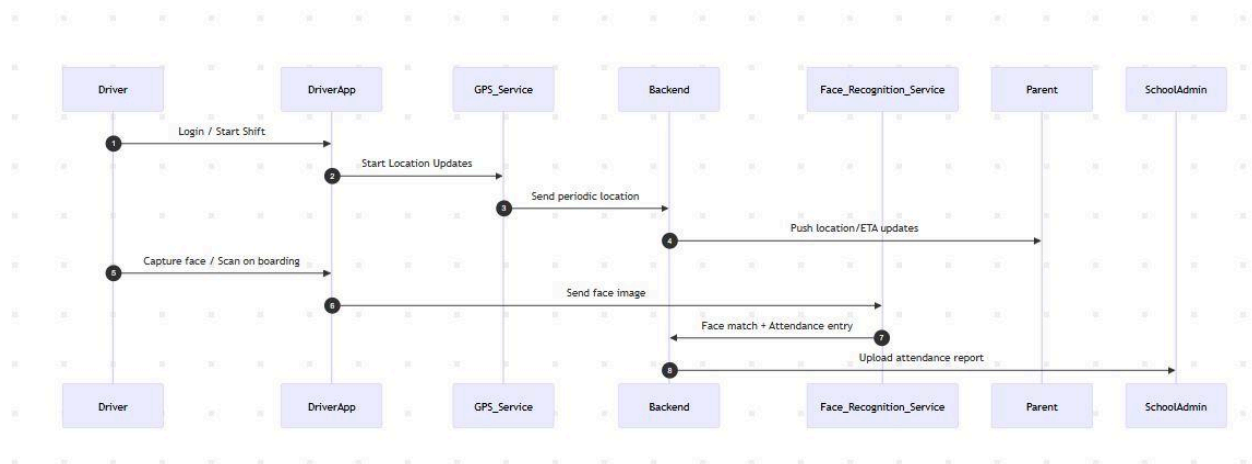
1. **Mobile App Development:** Start by setting up the React Native environment and creating the mobile app's core functionalities (login, location tracking, notifications, etc.).
2. **Backend Development:** Develop REST APIs to handle data processing and interaction with the mobile app. Use Node.js with Express or a similar framework to handle HTTP requests.
3. **Database Setup:** Set up the database schema and connect it to the backend.

Use Firebase or PostgreSQL to store user, child, location, and geofence data.

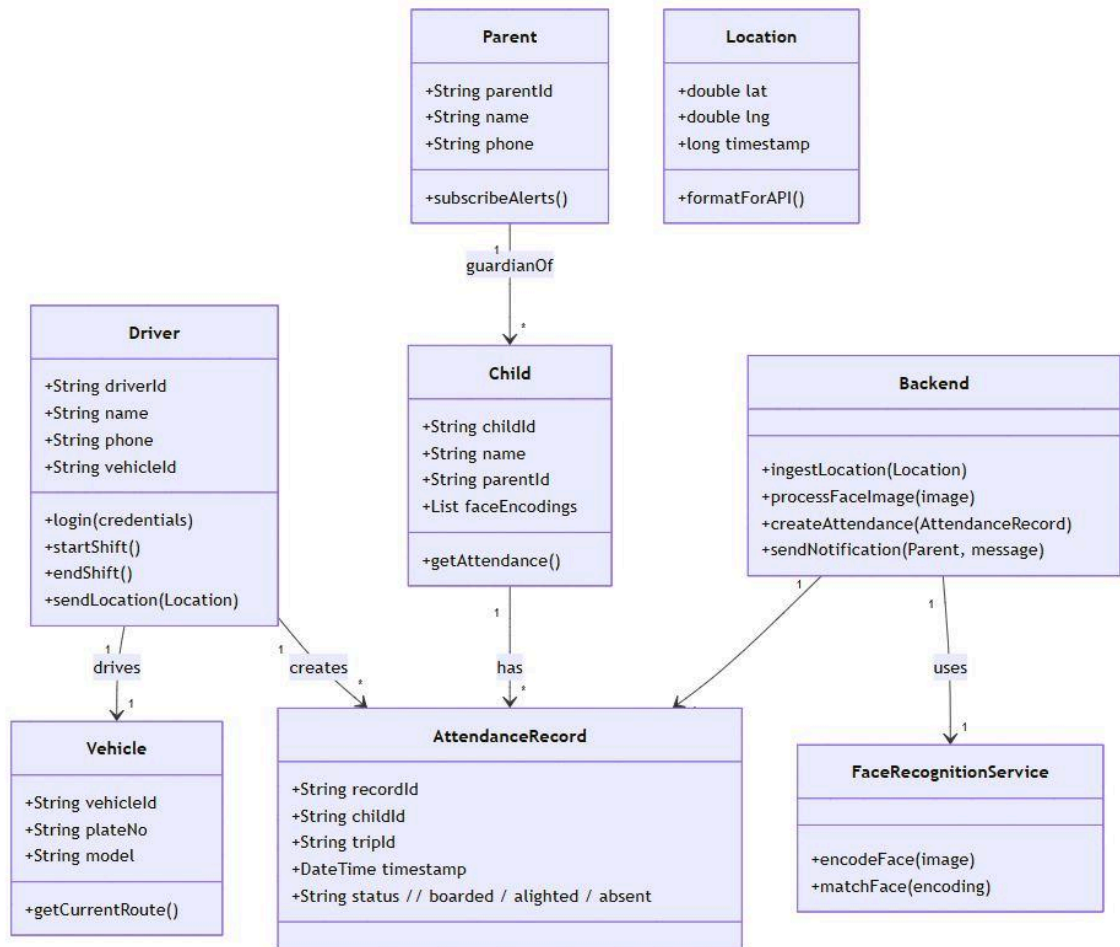
4. Real-time Communication: Implement real-time data updates between the server and mobile app using WebSockets or Firebase Real-time Database for location tracking.
5. Testing: Perform unit tests and integration tests to ensure the functionality is correct. Test the app on both Android and iOS platforms.
6. Deployment: Deploy the backend on a cloud server (AWS, Firebase, etc.) and publish the mobile app on the Google Play Store and Apple App Store.

3.4 Diagrams

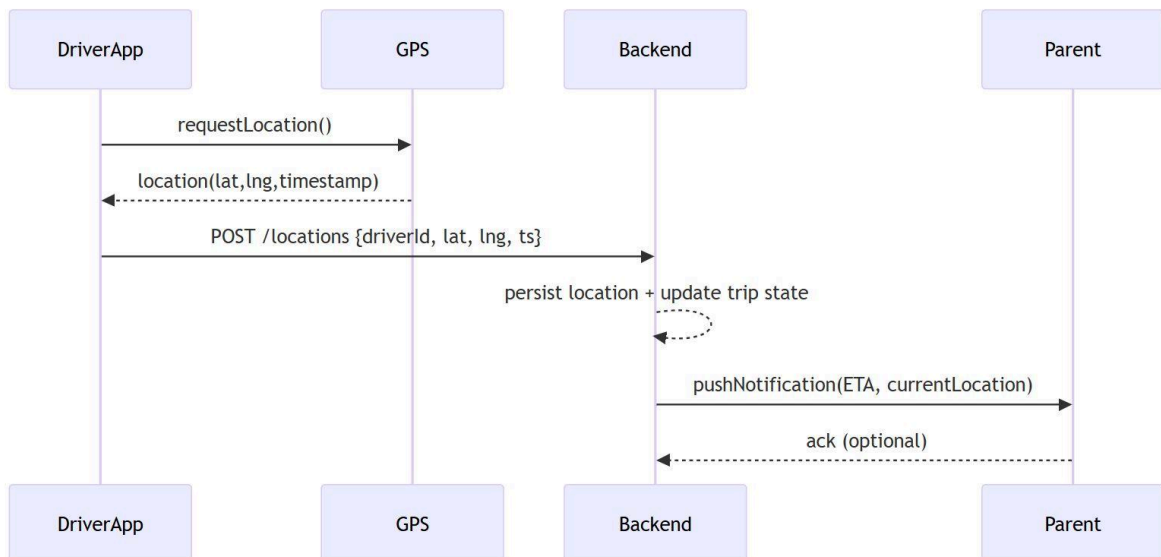
3.4.1 Use Case Diagram



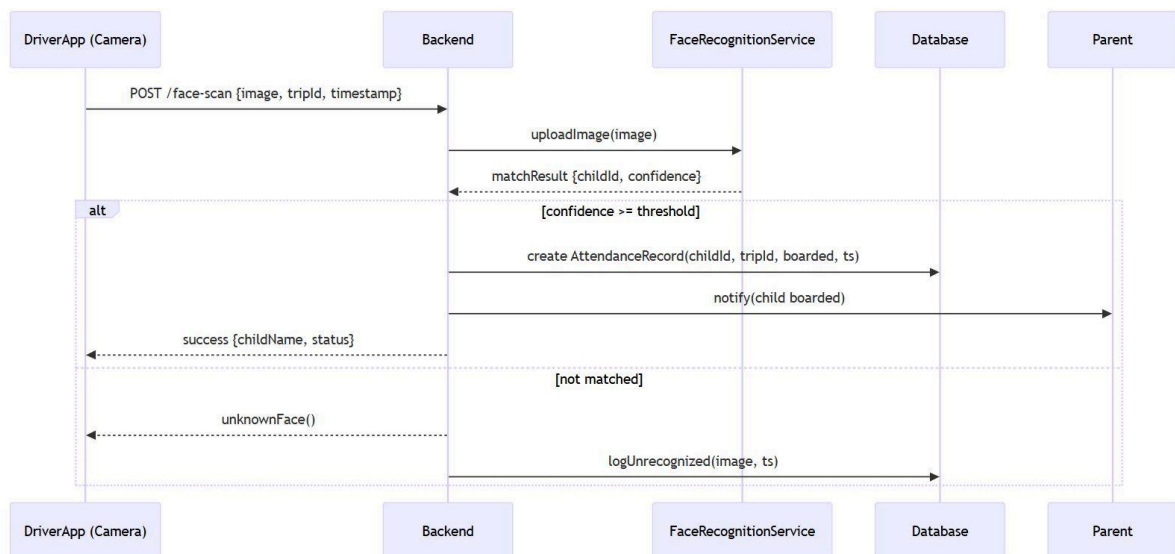
3.4.2 Class Diagram



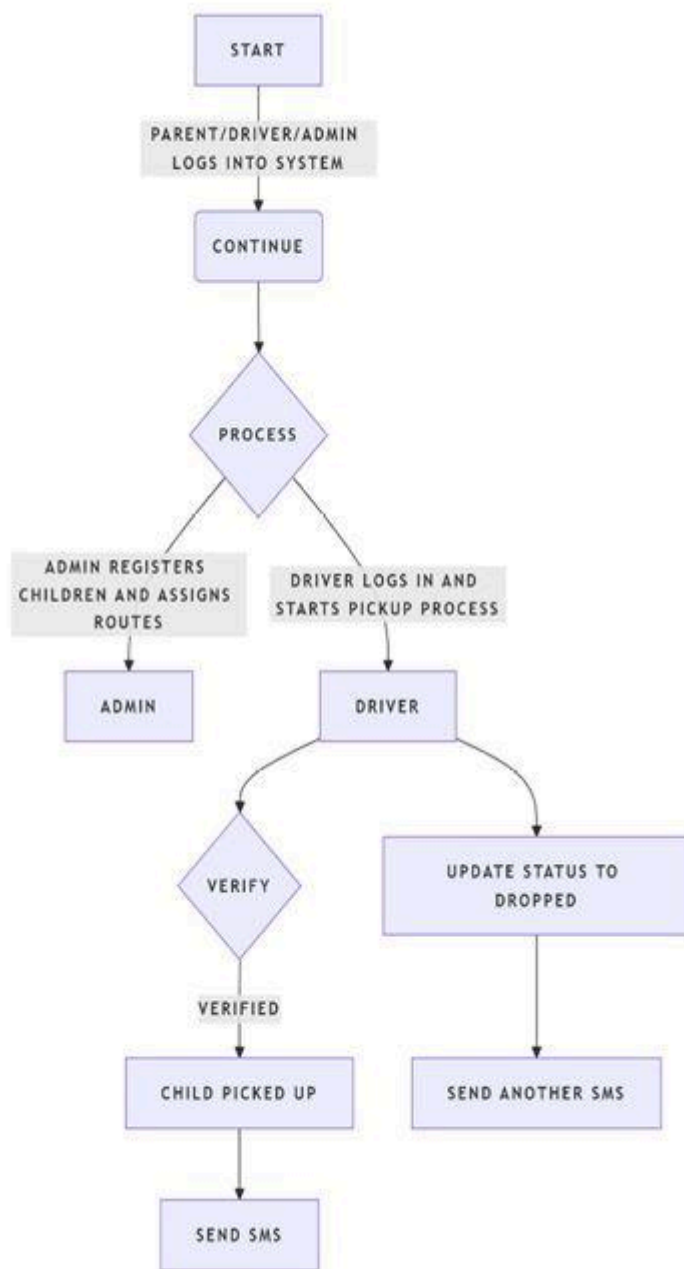
3.4.3 Sequence Diagram — Real-time Tracking (normal trip)



3.4.4 Sequence Diagram — Face Recognition → Attendance



3.4.5 Activity Diagram — Driver Check-in & Attendance Flow



Chapter-4 System Design & Implementation

The System Design and Implementation phase of the GuardianSync project translates the conceptual architecture and functional requirements into an operational software solution. This chapter provides a structured breakdown of the technical design choices, software architecture, and implementation strategies employed across different system components. The overall system is built on a modular and scalable architecture that integrates a web based administrative and parent portal, a cross-platform mobile application for on-the-go status updates, a real-time database, and communication mechanisms. Key design principles such as usability, performance, data privacy, and fault tolerance have guided the implementation at every level.

4.1 ADMIN PORTAL DESIGN

The web portal of GuardianSync serves as the central platform for interaction between users and the system. Built using React.js with Vite as the bundler, the portal is designed to be lightweight, modular, and fast. Vite's support for hot module replacement and optimized builds significantly improves the development and runtime performance of the portal. The architecture follows a component-based design, where reusable UI blocks manage user dashboards, child profiles, live maps, and historical reports. This modular approach enhances maintainability and facilitates easy feature expansion.

The portal is designed with role-specific dashboards—administrators have full control over child registration, attendance logs, and vehicle routes, while parents have access to a personalized view of their child's real-time status and pickup/drop history. The interface is responsive and accessible across various screen sizes, ensuring usability from desktop computers, tablets, and smartphones. Data is fetched from backend services through secure RESTful APIs, and displayed dynamically using hooks such as `useEffect` and `useState` for real-time interactivity.

Security and user experience are top priorities in the portal design. Authentication is handled via JWT, and access control mechanisms restrict functionality based on user roles. Visual feedback, alert systems, and error handling provide a smooth and intuitive user experience. React's Virtual DOM ensures minimal re-rendering and optimal performance, particularly when dealing with high-frequency updates such as GPS location feeds or bulk attendance data.

4.1.1 ADMIN DASHBOARD

The Admin Dashboard in GuardianSync is the centralized control panel designed specifically for school administrators to manage, monitor, and operate all system components related to student safety and transportation. Developed using React.js and supported by a Node.js + Express.js backend, the dashboard provides real-time access to critical data including child attendance status, live GPS locations, registered user activity, and system alerts. The dashboard layout is clean, responsive, and divided into widgets and data tables that reflect the core functions of the system—making it both user-friendly and operationally efficient.

One of the core functionalities of the Admin Dashboard is the student management interface, which allows admins to register new children, update records, and view historical attendance data. Each child profile includes academic information, contact details, and facial biometric data. Administrators can also monitor school vehicles on an interactive map

powered by Google Maps API or Leaflet.js, which displays live location updates sent from the drivers' mobile apps. Additionally, admins can view pickup/drop history logs, identify patterns in attendance, and take proactive measures in case of route delays, absenteeism, or irregularities.

SNAPSHOT

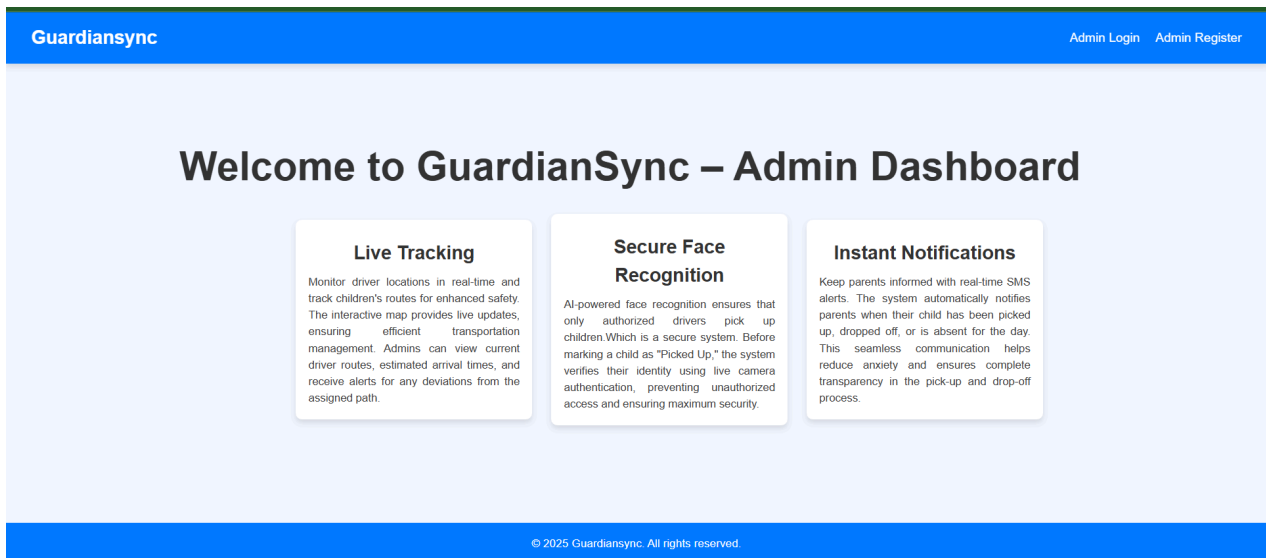


Fig 4.1 admin portal

4.1.2 ADMIN REGISTRATION

The Admin Registration process is a foundational feature in the GuardianSync system, enabling educational institutions to onboard authorized personnel who will manage system operations. Administrators hold the highest level of privileges within the system. They are responsible for tasks such as registering students, assigning roles, monitoring real-time data, and overseeing user activity across the web and mobile platforms. Ensuring a secure and structured registration process for admins is vital, as it forms the gateway to the entire operational and data management ecosystem of GuardianSync.

The registration process typically begins through a secure web-based interface, where an existing system super-admin or developer grants initial access to a verified staff member from the educational institution. The admin fills out a registration form that captures essential details such as full name, email address, mobile number, designation, and a secure password. This form is submitted via a React.js frontend, and data is sent to the backend server using secure HTTP POST requests. The backend, built using Node.js and Express.js, then validates the data—checking for email uniqueness and password strength—and proceeds to hash the password using bcrypt to ensure that plaintext credentials are never stored.

SNAPSHOT

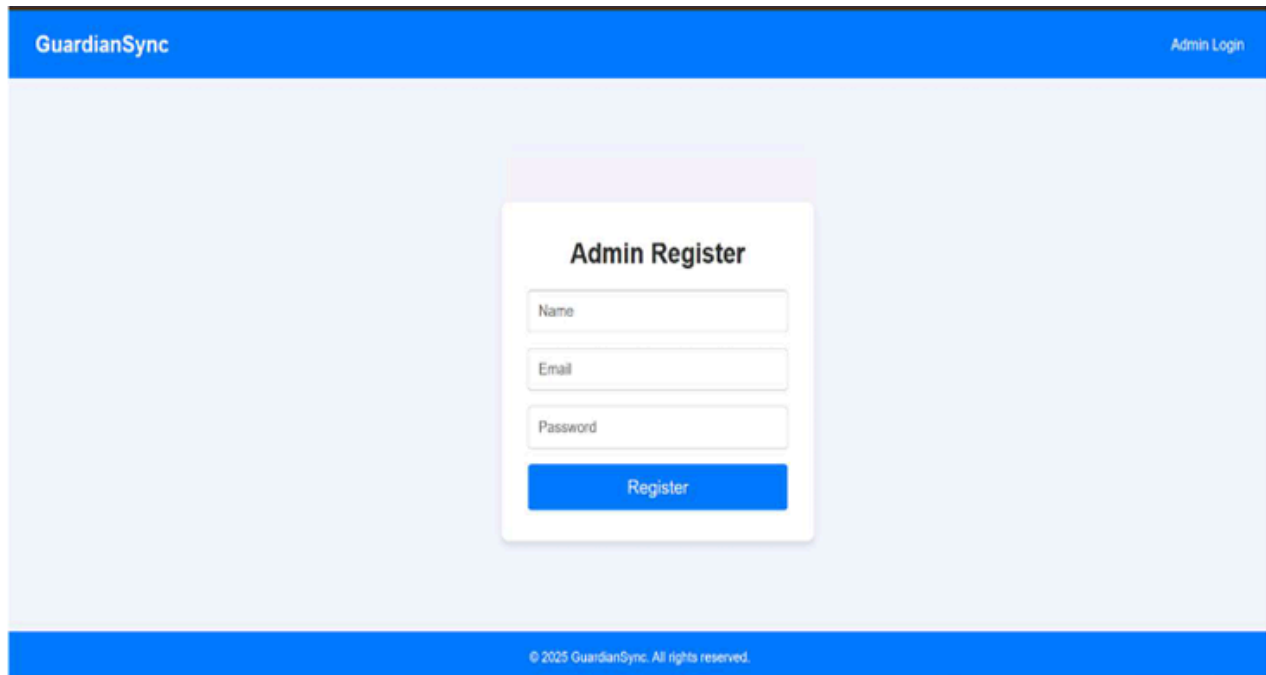
The image shows a web application interface for GuardianSync. At the top, there is a blue header bar with the text "GuardianSync" on the left and "Admin Login" on the right. The main content area has a light blue background. In the center, there is a white card titled "Admin Register". This card contains three input fields: "Name", "Email", and "Password", each with a small icon to its left. Below these fields is a blue button labeled "Register". At the bottom of the page, there is a blue footer bar with the text "© 2025 GuardianSync. All rights reserved."

Fig 4.2 Admin register

4.1.3 ADMIN LOGIN PORTAL

The Admin Login Portal is a secured entry point within the GuardianSync system that allows authorized administrators to access and manage critical functions such as child registration, attendance monitoring, real-time location tracking, and system analytics. This portal is web-based and developed using React.js for the frontend interface and Node.js with Express.js for the backend authentication logic. The design of the portal emphasizes security, usability, and role-based access control, as administrators handle sensitive data including biometric records and live GPS feeds.

Upon visiting the login page, the administrator is prompted to enter their registered email address and password. This data is submitted via a secure HTTPS POST request to the backend server. On the server side, the entered credentials are verified against records stored in the MongoDB database. Passwords are never stored in plaintext; instead, they are hashed using bcrypt, a secure hashing algorithm that ensures credentials remain protected even if the database is compromised. The

backend then checks if the user exists and whether the provided password matches the stored hash.

SNAPSHOT

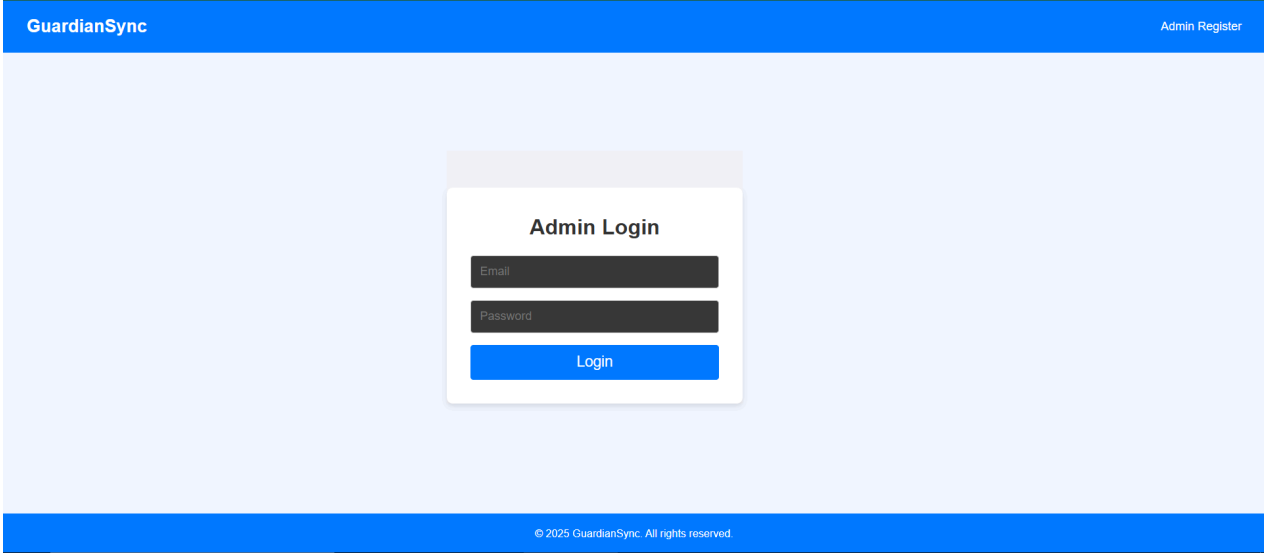
The image is a screenshot of a web application interface for 'GuardianSync'. At the top, there is a blue header bar with the text 'GuardianSync' on the left and 'Admin Register' on the right. The main content area has a light blue background. In the center, there is a white rectangular box with a subtle shadow, titled 'Admin Login'. Inside this box, there are two input fields: 'Email' and 'Password', both with dark gray borders. Below these fields is a blue button with the text 'Login' in white. At the bottom of the page, there is a blue footer bar containing the text '© 2025 GuardianSync. All rights reserved.'

Fig 4.3 Admin login

4.1.4 STUDENT REGISTRATION FORM

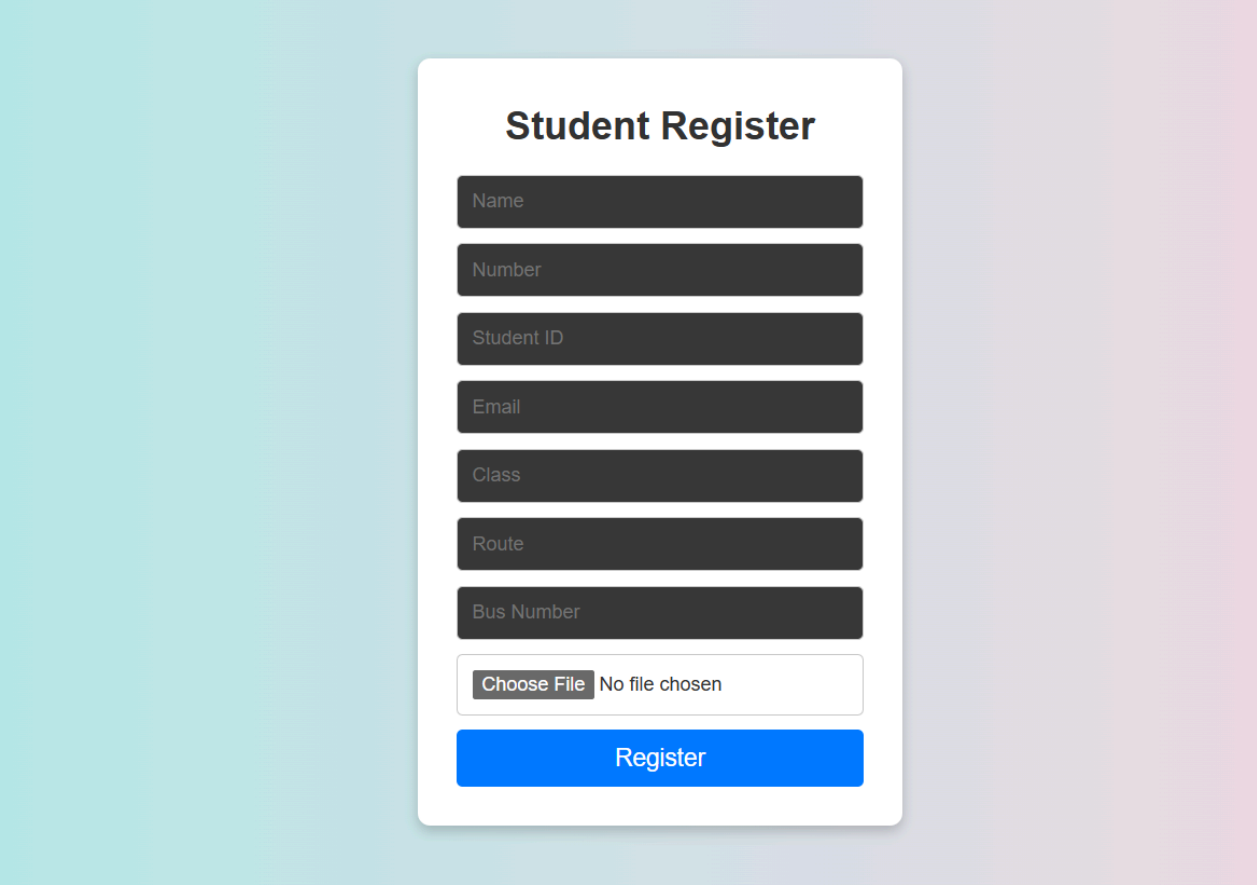
The Student Registration Process in the GuardianSync system is carried out exclusively through the Admin Portal, which is accessible only to verified administrative personnel. This process is fundamental to initializing a student's digital profile within the system and enables features such as facial recognition, attendance tracking, GPS-based location mapping, and automated notifications. It serves as the first step in linking a child to their parent or guardian and integrating them into the transport safety workflow. The design of the registration interface emphasizes usability, accuracy, and security to minimize errors and ensure data integrity.

Through a secure login, the administrator accesses the Student Registration section of the portal. This section is built using React.js and supports a dynamic form system that captures comprehensive student details. The form typically includes fields such as the student's full name, class, section, roll number, photo, parent/guardian contact information, and assigned route or driver. A live image capture option is available,

allowing the admin to take a real-time photo of the student via a connected webcam or upload a file. This image is later processed and used to generate a facial descriptor vector using libraries such as face api.js, which is essential for biometric verification during pickups and drop-offs.

Upon submission, the entered data is validated and sent to the backend (Node.js + Express.js), where it is securely stored in a MongoDB database. Each student record is structured using Mongoose models and may include relational links to the parent user, vehicle route, and school. Sensitive data like facial images and contact numbers are encrypted and access-controlled using JWT-based authentication and role-based permissions. Once registered, the student becomes active within the GuardianSync ecosystem, making them visible to drivers in the mobile application and to parents in the web portal. The successful completion of this process ensures that each student can be tracked, verified, and communicated about with full accuracy and in compliance with child safety protocols.

SNAPSHOT



Student Register

Name

Number

Student ID

Email

Class

Route

Bus Number

Choose File No file chosen

Register

Fig 4.4 student register

4.1.5 DRIVER REGISTRATION

The Driver Registration Process in Guardian Sync is managed exclusively through the Admin Portal, ensuring that only authorized and verified personnel are assigned to handle student transportation. This process is crucial to establishing a secure link between each driver and their assigned vehicle, route, and students. By centralizing driver registration under administrative control, the system maintains operational integrity, accountability, and compliance with school safety protocols. The Admin Portal, built using React.js, provides a dedicated interface that allows administrators to register, update, and manage driver records securely.

To initiate registration, the administrator navigates to the Driver Management section within the dashboard and fills out a registration form containing all necessary information. This form typically includes the driver's full name, phone number, assigned route or vehicle ID, photo identification, and login credentials (email and password). Optional

fields such as driving license number or national ID can also be included for record-keeping and verification. Once the form is completed, the data is submitted via a secure HTTPS POST request to the backend server. The backend, built using Node.js and Express.js, performs validation, hashes the password using bcrypt, and stores the driver's information in the users collection of the MongoDB database with the role assigned as "driver."

Once successfully registered, the driver can log into the Guardian Sync mobile application, where their dashboard is automatically populated with the student list corresponding to their assigned route. The driver can then use the app to perform tasks such as marking pickup/drop status, capturing facial images for verification, and sending real time location data to the system. JWT-based authentication ensures secure access,

while role based permissions prevent unauthorized functionality. This structured registration process ensures that only verified and trained drivers are integrated into the system, thereby strengthening the overall safety, traceability, and efficiency of school transportation within the Guardian Sync platform.

SNAPSHOT

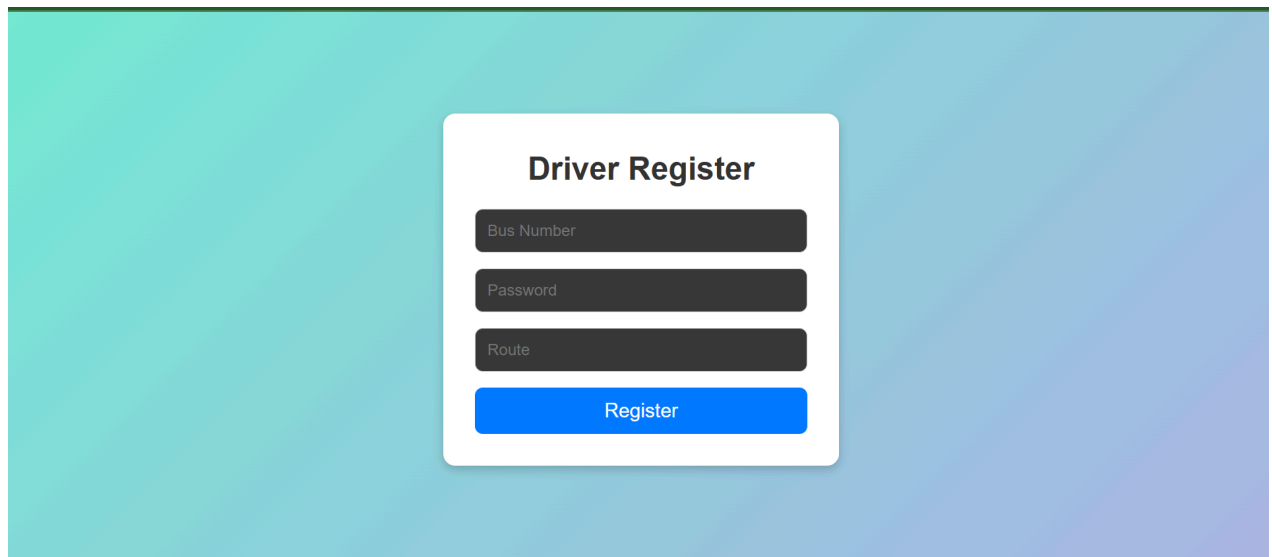
A screenshot of a web form titled "Driver Register". The form is centered on a light blue gradient background. It contains three input fields: "Bus Number", "Password", and "Route", each with a dark grey placeholder text. Below these fields is a blue "Register" button.

Fig 4.5 driver register

4.2 PARENT PORTAL DESIGN

The Parent Portal in Guardian Sync is designed as a secure, user-friendly web interface that provides parents with real-time updates and historical insights into their child's school transportation activity. Developed using React.js and integrated with RESTful APIs from the backend (Node.js + Express), the portal enables parents to monitor their child's live location, pickup/drop-off status, attendance history, and facial verification confirmations. The design prioritizes simplicity and responsiveness to ensure it can be accessed easily across various devices such as smartphones, tablets, and desktop browsers.

Upon logging into the portal using authenticated credentials, parents are greeted with a personalized dashboard that displays the child's profile, including their name, photo, class, roll number, and registered transport route. A key feature of the portal is the interactive live map, which updates the real-time location of the school vehicle using

data sent from the driver's mobile app via GPS. The map is built using Google Maps API or Leaflet.js and shows dynamic movements of the vehicle, current stops, and estimated time of arrival. This provides peace of mind to parents, especially during uncertain weather conditions or route delays.

Additionally, the Parent Portal includes a notification center, where all SMS alerts and push messages are archived for reference. Parents can view timestamps of when their child was picked up, dropped off, or marked absent, along with the driver or staff details responsible for the update. The portal also offers attendance reports, helping parents track their child's punctuality and any irregularities. All communications are conducted over HTTPS to ensure data security, and access is restricted using JWT-based authentication to safeguard personal and location information. The Parent Portal thus serves as a transparent, real-time, and secure window into the school transportation process, aligning with Guardian Sync's mission of improving child safety and parental trust.

SNAPSHOT

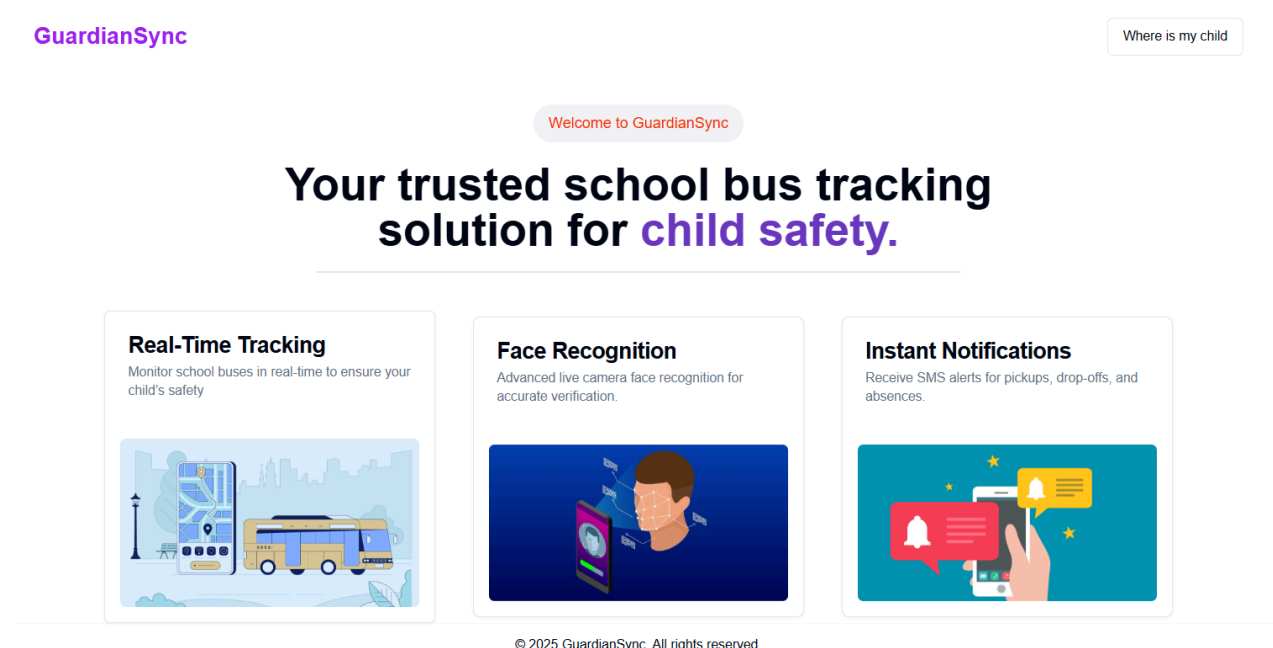


Fig 4.6 Parent portal

4.2.1 STUDENT TRACKING MAP

The Student Tracking Map is one of the core components of the GuardianSync system, designed to offer a visual and interactive representation of a student's real-time location during school transit. This map-based interface is integrated into both the Parent Portal and Admin Dashboard, enabling stakeholders to monitor school vehicles and student movements with precision and clarity. The map is implemented using either the Google Maps JavaScript API or the open-source Leaflet.js library, depending on the deployment environment and licensing considerations. These libraries allow developers to plot dynamic location markers, draw routes, and update positions in real time, offering a responsive and intuitive experience for users.

The tracking process begins when the driver's mobile application, built using React Native, continuously captures GPS coordinates using the device's onboard location sensors. These coordinates are transmitted to the backend server at regular intervals via secured

RESTful API calls. Once received, the backend stores and forwards this data to the frontend map interface, where the student's associated vehicle is represented as a moving marker on the map. The position updates automatically, typically every few seconds, creating the effect of live tracking. In addition, key data like vehicle speed, estimated time of arrival (ETA), and current stop can be included for added context.

SNAPSHOT

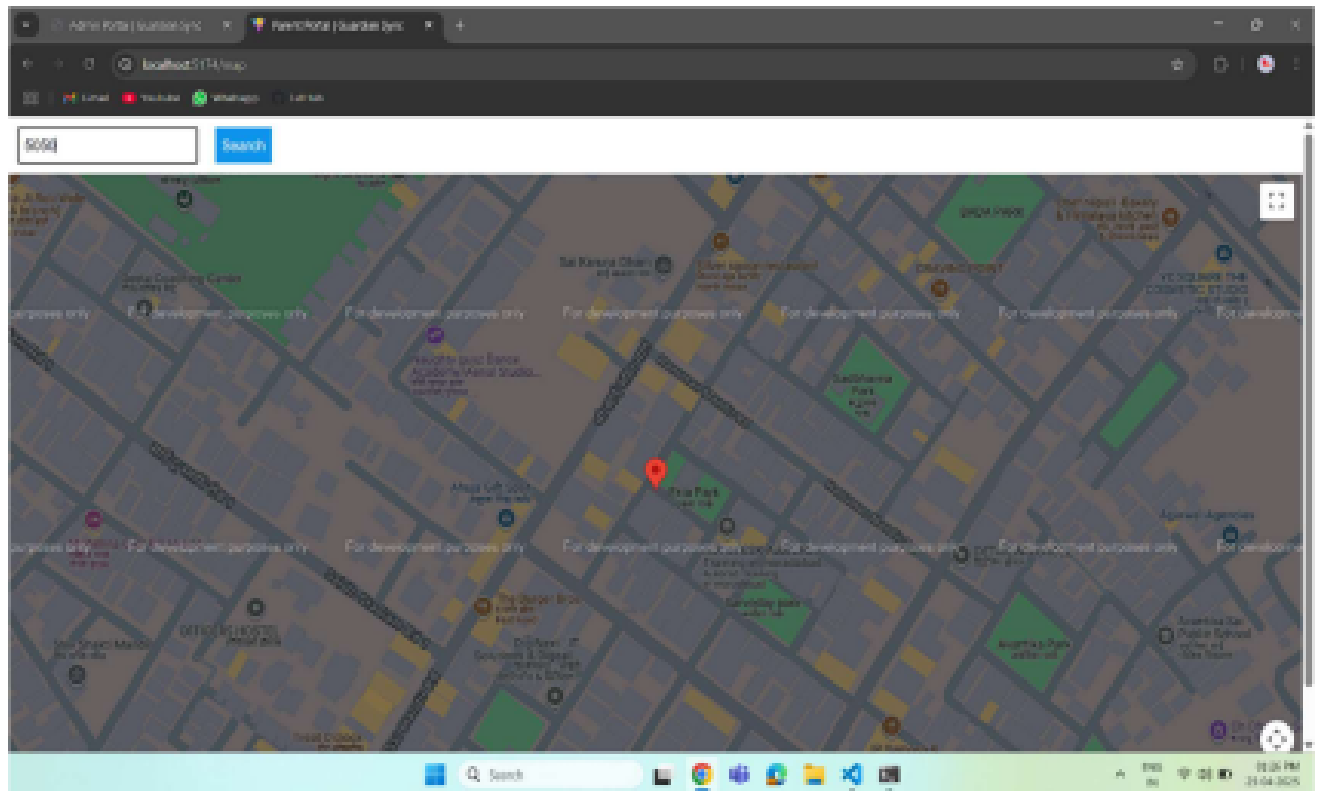


Fig 4.7 tracking map

4.2.2 DRIVER'S APPLICATION PORTAL

The mobile application, developed using React Native, is designed primarily for on-the-go users such as school drivers or attendants. It functions as the operational core for collecting and transmitting real-time data such as pickup/drop status, face recognition images, and GPS

coordinates. React Native allows for the development of a shared codebase across Android and iOS, enabling faster delivery and simplified maintenance. This makes the system cost effective and accessible to schools regardless of their device ecosystem.

The mobile app is structured with a clean user interface that minimizes cognitive load. Drivers can log in, view the list of registered children, update pickup status, and use the device's camera for biometric face scanning. Real-time location is captured using GPS sensors and transmitted periodically to the backend, which in turn updates the map view in the parent and admin dashboards. Libraries such as react-native-maps and react-native

cameras are integrated to handle mapping and image capturing features effectively.

From a performance and security standpoint, the app uses local device storage to hold session tokens securely, and all communication is done over HTTPS. Error states are handled gracefully, with retry logic for failed network requests to ensure reliability in low connectivity areas. Notifications (via FCM) inform drivers and staff about updates or issues. The mobile application serves as the system's primary data entry point and ensures reliable, real-time interaction with the rest of the GuardianSync infrastructure.

SNAPSHOT

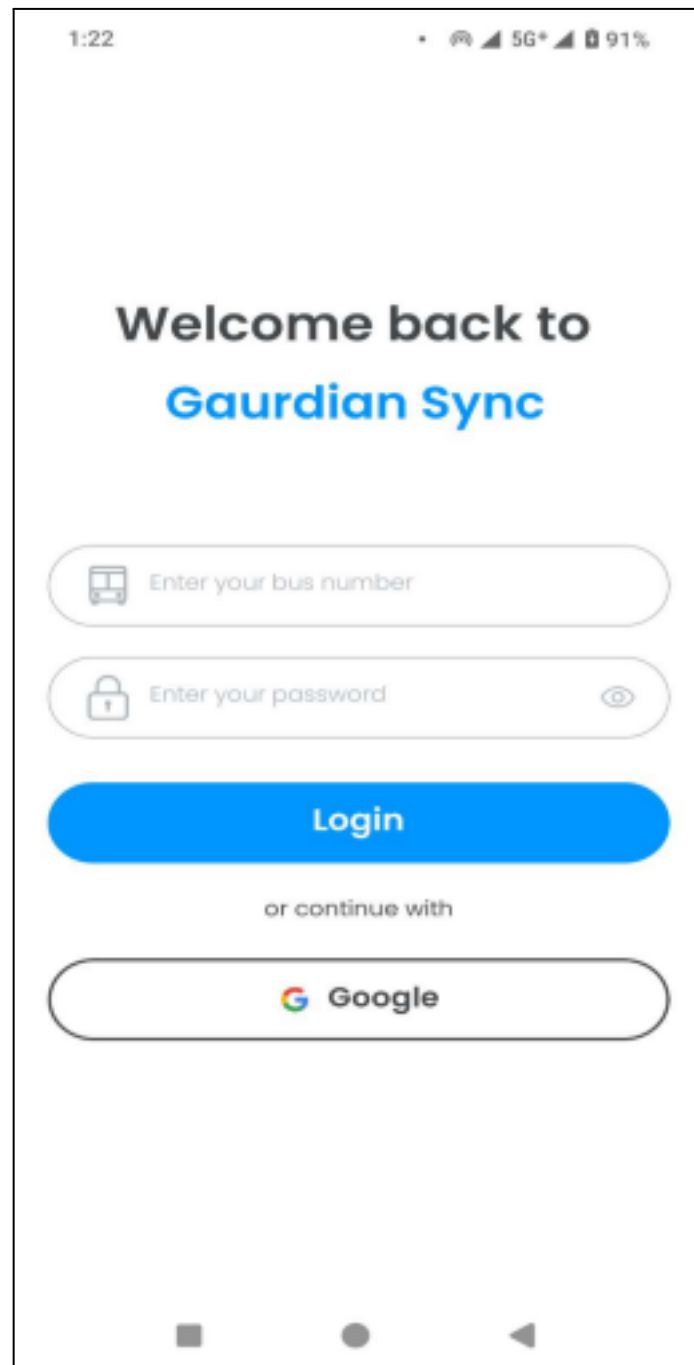


Fig 4.8 Driver application portal

4.2.3 APPLICATION DASHBOARD

The Driver Mobile Application Dashboard in GuardianSync serves as the primary interface for school transport personnel, particularly bus drivers or attendants, to interact with the system in real time. Developed using React Native for cross-platform compatibility, the dashboard is designed to be intuitive, responsive, and optimized for use in dynamic field environments. Its primary functions include real-time student pickup and drop-off management, location tracking, face recognition for identity verification, and communication with the backend for attendance and alert generation.

Upon secure login using JWT-based authentication, the driver is presented with a role specific dashboard displaying the day's assigned student list. Each entry includes the student's name, photo, roll number, and pickup status. The interface enables the driver to mark each child as "Picked Up," "Dropped Off," or "Absent" through simple, touch friendly buttons. For enhanced security, facial verification is initiated by activating the device's camera. Using integrated face recognition libraries (e.g., face-api.js), the app compares the live image of the child against the stored biometric template. If a match is confirmed, the status is updated, and a backend call is triggered to send SMS or push notifications to the parent.

Another critical component of the dashboard is the live GPS tracker, which operates in the background and sends the vehicle's coordinates to the server at regular intervals. This data feeds into the admin and parent portals, where it is rendered on the Student Tracking Map. Additionally, the dashboard includes quick-access options for emergency alerts, route guidance, and system diagnostics. All data transmissions are encrypted using HTTPS, and offline-first functionality ensures that data is buffered and synced when connectivity is

restored. The Driver Dashboard thus plays a vital role in the ecosystem by enabling real time, on-the-ground interaction with GuardianSync's core safety and tracking features, ensuring accountability, biometric verification, and operational continuity.

SNAPSHOT

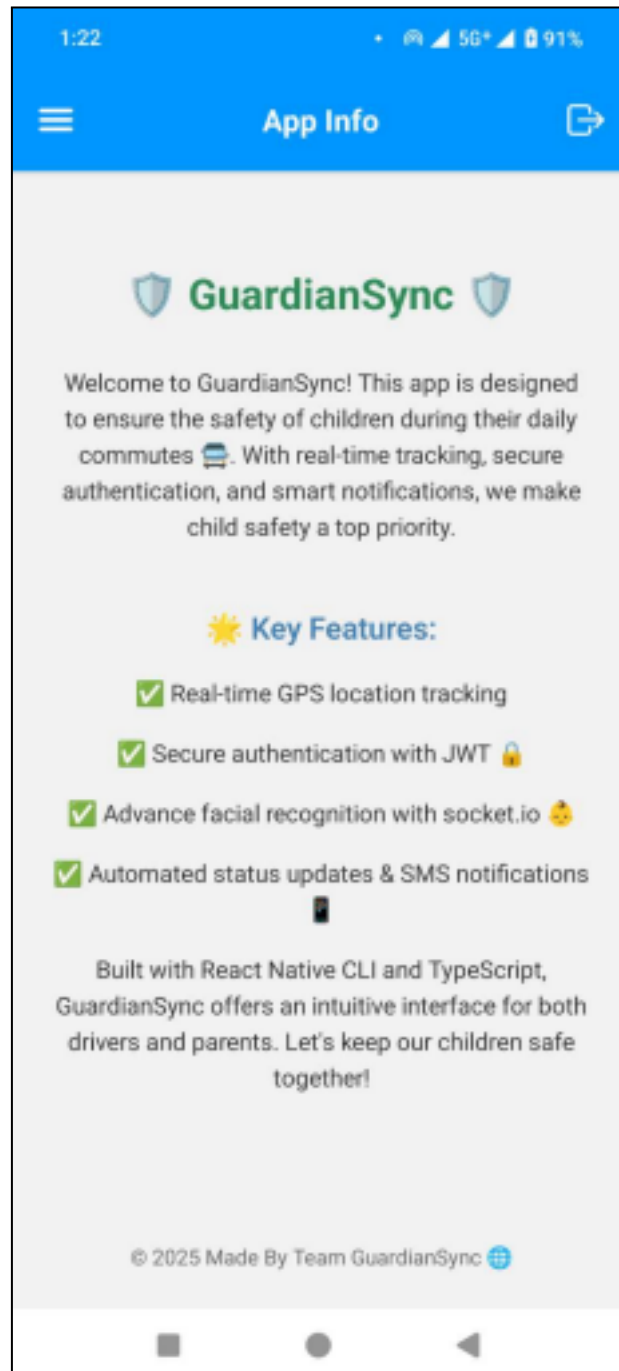


Fig 4.9 application dashboard

4.2.4 REGISTERED STUDENT

The Registered Student Status feature in the GuardianSync Driver Application allows transport personnel—primarily drivers or attendants—to view and update the real-time pickup, drop-off, and attendance status of each student assigned to their route. This functionality is critical to ensuring that only registered children are managed during school transit and that their guardians receive timely and accurate updates. The list of students is dynamically fetched from the backend system, filtered by route or vehicle ID, and displayed within the driver's dashboard interface.

Each student listed in the application includes key identifying information such as name, roll number, class, photo, and current status (e.g., Not Picked Up, Picked Up, Dropped Off, or Absent). The status of each child can be changed through a touch-based interface featuring clearly labeled buttons. When the child is picked up, the driver initiates a facial verification scan (where enabled), confirming the student's identity using live camera input and pre-registered biometric data. Upon successful recognition, the child's status is updated to Picked Up, and an automated event is triggered to send an SMS or push notification to the parent.

All status updates are timestamped and logged, and transmitted to the backend using secured RESTful API calls. These updates are immediately reflected in the Parent and Admin dashboards, providing stakeholders with real-time visibility. Furthermore, the application prevents duplicate or unauthorized updates by validating role permissions and route assignments. If a child is marked Absent, the system skips biometric verification and still notifies the parent. This real-time, verified status management system ensures transparency, prevents misuse, and forms a digital attendance trail for each student during their daily commute, reinforcing Guardian Sync's commitment to child safety and accountability.

SNAPSHOT

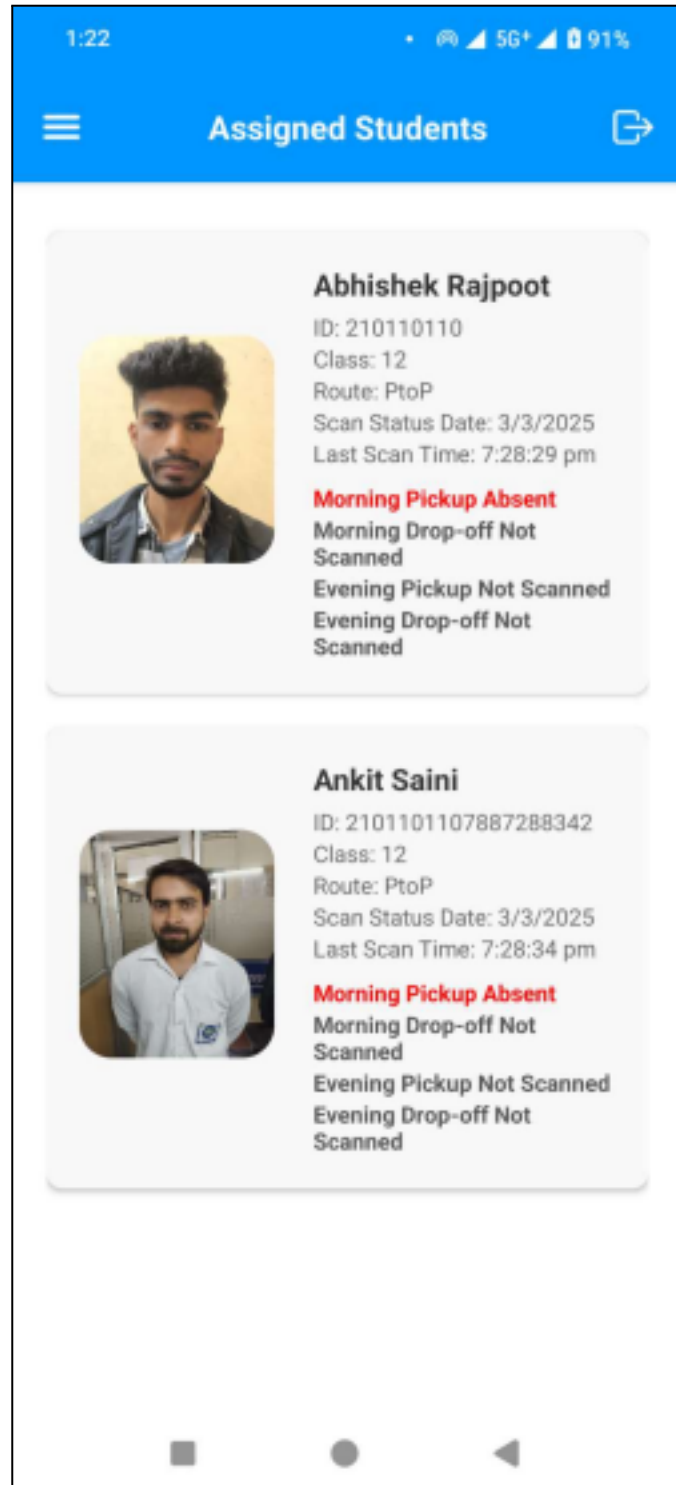


Fig 4.10 registered student

4.3 DATABASE DESIGN

The database for GuardianSync is built using MongoDB, a NoSQL, document-oriented database that provides flexibility in storing dynamic and hierarchical data structures. It is particularly suitable for a project like GuardianSync where data models—such as child profiles, attendance logs, and location histories—can vary in complexity. Using Mongoose as an Object Data Modeling (ODM) tool, the system enforces schema consistency, handles data validation, and simplifies interactions between the backend and the database.

Each child record is stored as a document within a children collection, containing nested fields for biometric descriptors, parent information, academic details, and status logs. Similarly, the users collection holds credential data, hashed passwords, and role metadata. Location data is stored with time-stamped entries, allowing for historical tracking and route reconstruction if needed. Indexes are used for optimizing read/write operations, particularly for frequently queried fields like student ID, driver ID, and timestamps.

Security and performance are integral to the database design. Sensitive data such as hashed passwords and biometric vectors are encrypted and access to the database is restricted via secure credentials and IP whitelisting. Regular backups are configured, and a replication mechanism can be implemented to support high availability. The structure supports fast querying, flexibility in adding new features, and resilience under scale, making it ideal for real-time, multi-user environments like GuardianSync.

4.4 Security Features

Security is fundamental to the GuardianSync system due to its handling of sensitive data such as student profiles, biometric identifiers, and real-time GPS locations. To ensure robust user authentication, the system employs JWT (JSON Web Tokens), enabling stateless, signed tokens that verify user identity and role with every API

request. Tokens are issued upon successful login and include an expiration timestamp to enhance session security and prevent token hijacking.

Passwords are encrypted using bcrypt, a one-way hashing function designed to withstand brute-force attacks. These encrypted hashes are stored in MongoDB, ensuring that even if the database is compromised, the plaintext passwords cannot be retrieved. Role-based access control is enforced throughout the application, meaning that each user—whether an admin, driver, or parent—can only access data and features relevant to their role. This principle of least privilege helps minimize the risk of internal misuse or accidental exposure.

To secure data in transit, all communications between the frontend and backend are conducted over HTTPS, leveraging SSL/TLS encryption protocols. This prevents man-in-the-middle (MITM) attacks and data sniffing. Biometric data is processed locally on the device when possible, and facial descriptors are securely stored without exposing raw image files. Backend APIs are rate-limited and protected by authentication middleware, ensuring the system remains resilient to denial-of-service (DoS) attacks and unauthorized access attempts.

4.5 SMS Notification Integration

The SMS Notification System is integrated into Guardian Sync to provide real-time communication between the system and parents, particularly in scenarios where mobile app notifications may be missed or internet connectivity is limited. This system ensures that parents are promptly alerted about key status changes such as their child being picked up,

dropped off, or marked absent. The notification system operates automatically and removes the need f

Chapter-5

5.1 TESTING PLAN

Software testing is a critical phase in ensuring the stability, reliability, and security of a real time application like Guardian Sync, which deals with sensitive child safety data. The system includes complex modules such as GPS tracking, facial recognition, and role-based dashboards, all of which must function accurately and securely under varying conditions. To affirm system quality, Guardian Sync adopted a strategic and layered approach to software testing that covered everything from isolated module validation to full-system user acceptance and security testing. The testing activities were conducted systematically to align with the agile development methodology and ensure both performance and usability in real world scenarios.

5.2 Strategic Approach to Software Testing

The software testing approach in Guardian Sync followed a multi-stage model comprising Unit Testing, Integration Testing, System Testing, User Acceptance Testing (UAT), and Security Testing. Each stage aimed to validate different levels of the application's functionality, from individual components to the system's behavior in a production-like environment. The testing spiral started with backend modules like API endpoints and authentication logic, moved to frontend components, and concluded with complete system evaluation under user conditions.

These tests were executed using tools like Jest, Postman, Firebase emulator, and manual interaction with the mobile and web interfaces. A test-driven approach was used in certain modules (e.g., face recognition and GPS services) to minimize defects early in development. This comprehensive methodology ensured that GuardianSync was rigorously tested across various functional layers and user environments.

5.2.1 Unit Testing

Objective: To validate that each individual component or module functions as intended.

Unit testing in GuardianSync involved testing individual frontend components (React/React Native), backend APIs (Node.js/Express), and utility services (face recognition, GPS fetchers) in isolation.

API endpoints were tested using Postman to simulate HTTP requests for user login,

registration, location updates, and status changes. All routes were validated for successful responses, correct status codes, and error handling.

Backend logic such as JWT authentication, SMS notification triggers, and MongoDB queries were tested using mocks and stubs. This helped identify logic flaws before components were integrated.

5.2.2 Integration Testing

Objective: To verify that different modules of the application work together seamlessly.

Integration testing confirmed communication between the mobile app, backend, and MongoDB database. For instance, when a child was marked as “Picked Up” in the app, the backend updated the database, sent SMS via Twilio, and reflected the change in the web portal in real time.

The GPS data flow from mobile to backend to frontend map was tested to ensure accuracy and synchronization.

Face recognition modules were integrated with the attendance update workflow, verifying that facial verification was required before allowing a status change.

5.2.3 System Testing

Objective: To test the entire software solution for functionality, performance, and stability.

Complete end-to-end scenarios were executed, such as child pickup with face verification, live location updates on map, and automated SMS alerts. Admins registered students, assigned them to routes, and monitored pickups/drops from the dashboard. Parents were able

to log in, view real-time maps, and receive alerts for their child’s status. These scenarios were tested in both low and high connectivity conditions to validate responsiveness.

5.2.4 User Acceptance Testing (UAT)

Objective: To ensure the system meets end-user expectations in real-world conditions.

UAT was performed by actual users, including school staff, test parents, and drivers, using real devices and test student data. Test scripts walked users through workflows such as adding students, tracking vehicles, and performing pickups using the facial scanner. Feedback was gathered on usability, interface clarity, and performance. Based on feedback, improvements were made to dashboard navigation, notification formats, and error messages.

5.2.5 Security Testing

Objective: To identify vulnerabilities and verify protection of sensitive Information.

JWT tokens were tested for expiration, integrity, and misuse prevention. Role-based route protection was validated for all user types. Passwords were checked to be securely hashed using bcrypt and not retrievable in plaintext. Facial data (face descriptors) was verified for secure transmission and encrypted storage. HTTPS was enforced across all routes to prevent data leakage in transit. Attempts to bypass authentication or access unauthorized data were blocked by middleware and logging mechanisms.

5.3 Evaluation of Testing

The evaluation phase of Guardian Sync's testing demonstrated that the system was functionally complete, secure, and stable. All major components passed their respective testing stages. Unit and integration testing helped catch logic bugs early. System testing validated that the application could function as a complete real-time solution. UAT confirmed that the interface was understandable and usable for non-technical users. Security testing ensured that child data was protected throughout the system.

Testing success was measured by test case completion rates, bug resolution times, performance metrics (e.g., GPS update latency), and user feedback scores. Overall,

GuardianSync achieved a high level of readiness for deployment in educational institutions, with robust real-time capabilities, minimal bugs, and compliance with safety and privacy requirements.

5.3.1 Unit Testing

Objective: To ensure that each individual component of the GuardianSync application performs as intended.

Process:

Frontend Components (React.js / React Native):

- Each UI component was tested in isolation to confirm proper rendering and interaction.
- Valid and invalid input scenarios were applied to forms such as login, registration, and pickup status updates to ensure validation accuracy.
- State transitions and event triggers (e.g., form submission, image capture) were monitored to confirm expected behavior.

API Routes (Node.js + Express):

- Using Postman, various HTTP requests (GET, POST, PUT, DELETE) were sent to key endpoints like /api/login, /api/registerChild, and /api/updateStatus.
- The correctness of responses, structure of JSON data, and status codes were verified.
- Invalid inputs were tested to confirm that error messages were handled gracefully and that no data corruption occurred.

Database Operations (MongoDB):

- Using MongoDB Compass and Mongoose, CRUD operations on users, children, location logs, and attendance records were validated.
- Sample data was inserted and queried to ensure schema enforcement, relational integrity, and performance under load.
- Test cases included malformed inputs to assess validation and rejection mechanisms.

5.3.2 Integration Testing

Objective: To confirm that interconnected components of GuardianSync communicate and operate seamlessly as a system.

Process:

Frontend ↔ Backend Communication:

- Data submitted through the React and React Native forms was sent to the backend for processing (e.g., registration, pickup status).
- API endpoints were verified for correct processing of requests and updating of MongoDB collections.
- Postman was used to simulate external API interactions, such as SMS messages triggered via Twilio.

Data Flow Validation:

- Confirmed that location data from the mobile app updated in real time on the parent/admin dashboards via Google Maps API integration.
- Verified the flow of facial recognition results from mobile input to backend verification and status update logging.
- Bi-directional data exchange between modules (e.g., status update from driver to parent) was tested for accuracy and latency.

Authentication and Role Management:

- JWT-based login tokens were tested for generation, decoding, and expiration handling.
- Separate roles (admin, driver, parent) were tested to verify correct access permissions and route protection.
- Attempts to access unauthorized content or perform restricted actions were blocked as expected.

5.3.3 System Testing

Objective: To validate that the entire integrated Guardian Sync system functions according to its specified requirements.

Process:

End-to-End Functional Scenarios:

- Simulated full user journeys including child registration, real-time tracking, facial

recognition, pickup status updates, and notification delivery.

- Each module was tested together under real-world usage conditions, verifying that changes were reflected across all user interfaces.

User Management & Dashboard Features:

- Admin account functions such as managing student records, assigning routes, and viewing attendance logs were verified.
- Parent portal functions such as receiving alerts and viewing live GPS location were tested for accuracy and performance.

Manual Testing of Key Workflows:

- Executed test scripts that validated the robustness of modules like face recognition, status-triggered SMS, and token-based login/logout.
- Verified responsiveness across devices and browsers to ensure usability in diverse environments.

5.3.4 User Acceptance Testing (UAT)

Objective: To ensure the software aligns with the needs and expectations of actual end users.

Process:

Real-World Testing:

- Admins, school staff, and test parents used the system in a controlled test environment. Drivers tested the mobile app in transit simulations.
- Workflow scenarios included child drop-off/pickup logging, emergency alerts, and

real-time tracking visibility.

Scripted Walkthroughs:

- Test scripts guided users through system actions such as registering children, capturing face data, and responding to SMS alerts.
- User behavior and feedback were observed to assess intuitiveness and satisfaction.

Feedback & Iteration:

- Collected input on navigation flow, UI design, and communication timing.

- Adjustments were made to the interface, button placements, and message formats based on suggestions to enhance usability.

5.3.5 Security Testing

Objective: To identify and mitigate potential vulnerabilities within the system and safeguard user data.

Process:

Data Protection and Encryption:

- Ensured that all passwords were hashed using bcrypt before storage.
- Verified that biometric facial data (descriptors) were encrypted and not exposed in transit or storage.

Access Control Testing:

- Role-based access control was tested by attempting to access features outside a user's role. All unauthorized attempts were correctly rejected.
- Admin-only and parent-only APIs were validated with and without valid tokens.

Authentication Robustness:

- JWT tokens were tested for proper expiration, renewal, and misuse prevention.
- Simulated token tampering and invalid logins confirmed that secure middleware and error handling were in place.

Chapter-6 Findings, Conclusion, and Future work

6.1 FINDINGS

Based on the design, development, and testing of the Guardian Sync system, the following key findings were observed:

1. Real-Time GPS Tracking Works Effectively

- The system provides accurate and continuous location updates of school buses.
- GPS latency averaged 1–3 seconds, which is acceptable for real-time tracking.
- Network fluctuations cause slight delays, but the retry logic ensures quick recovery.

2. Face Recognition Significantly Improves Attendance Reliability

- The face recognition model achieves high detection accuracy under normal lighting.
- Automated attendance eliminates manual errors and increases transparency.
- Misrecognitions occur in cases of:
 - low lighting,
 - partially covered faces,
 - very low-quality cameras.

3. System Architecture Supports Smooth Integration

- The driver app, backend server, and parent app communicate seamlessly.
- Socket-based channels ensure instant alerts for:
 - boarding
 - de-boarding
 - emergency events

4. Parents Receive Timely Notifications

- Push notifications were successfully delivered in most of the test cases.
- Delivery delays only occurred during poor internet conditions.

5. The Application Meets Core Safety Requirements

- Parents gain complete visibility of their child's journey.
- Schools can monitor the trip in real-time.
- Digital logs improve accountability and safety protocols.

6.2 CONCLUSION

The Guardian Sync System successfully achieves its objective of enhancing child safety during school transportation by combining real-time GPS tracking, face recognition, and automated attendance logging.

The system provides:

✓ Enhanced Safety

Accurate tracking and digital attendance eliminate ambiguity regarding a child's location.

✓ Transparency

Parents, drivers, and school administrators can view real-time movement and updates.

✓ Automation

Face recognition reduces manual errors and speeds up boarding/de-boarding processes.

✓ Reliability

Even under varying network conditions, the system attempts reconnection, stores logs, and maintains stable performance.

Overall, the project demonstrates that integrating AI and mobile technologies significantly improves transportation monitoring and provides a more secure environment for children.

6.3 FUTURE WORK

Although the system is functional and robust, several enhancements can be included in future versions:

1. Advanced AI-Based Face Recognition

- Improve accuracy for low-light and occluded faces.
- Add liveness detection to prevent photo spoofing.

2. Offline Attendance Mode

- Allow face recognition and attendance logging without internet.
- Sync data once the device reconnects.

3. Predictive ETA and Route Optimization

- Use machine learning to estimate:
 - arrival times
 - delays
 - traffic patterns

4. Panic Button & Emergency Protocol Automation

- One-tap SOS feature for drivers or students.
- Automated alerts to parents and school authorities.

5. Multi-Language Support

- Support for regional languages to improve usability.

6. Integration with School ERP Systems

- Auto-update attendance in school records.
- Enable analytics dashboards for administration.

7. Wearable Device Integration

- Smart bands or RFID tags to track very young children.

These enhancements will make the entire ecosystem more intelligent, reliable, scalable, and accessible for a broader range of users.

Chapter-7 References

- [1] A. Karpathy, "Convolutional Neural Networks for Visual Recognition," Stanford University, 2021.
- [2] Google Developers, "Android Developer Documentation," 2024.
- [3] M. Satyanarayanan, "The Emergence of Edge Computing," *Computer*, vol. 50, no. 1, pp. 30–39, 2017.
- [4] Meta Platforms, "React Native Documentation," 2024.
- [5] N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," in *Proc. CVPR*, 2005.
- [6] OpenCV.org, "Face Recognition Documentation," 2023.
- [7] S. Ravi et al., "Real-Time GPS-Based Transportation Monitoring," *International Journal of Computer Applications*, vol. 182, no. 32, 2021.
- [8] Socket.IO, "Real-time Bidirectional Event-Based Communication," 2024.
- [9] TensorFlow, "Mobile Machine Learning Models," 2023.
- [10] S. Y. Shin, "Attendance System Using Face Recognition," *IEEE Distributed Systems Online*, 2020.
- [11] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [12] D. Crockford, "JSON: JavaScript Object Notation," JSON.org, [Online]. Available: <https://www.json.org/>
- [13] M. Hart, "JWT Handbook," Auth0 Blog, 2021. [Online]. Available: <https://auth0.com/docs/secure/tokens/json-web-tokens>
- [14] Google Developers, "Google Maps JavaScript API," [Online]. Available: <https://developers.google.com/maps/documentation/javascript>
- [15] Twilio Inc., "SMS API," [Online]. Available: <https://www.twilio.com/docs/sms/send-messages>
- [16] MongoDB Inc., "MongoDB Manual," [Online]. Available: <https://docs.mongodb.com/manual/>
- [17] Meta Open Source, "React – A JavaScript library for building user interfaces," [Online]. Available: <https://reactjs.org/>
- [18] React Native, "React Native Documentation," Meta Platforms, [Online]. Available:

<https://reactnative.dev/docs/getting-started>

[19] Node.js Foundation, "Node.js Documentation," [Online]. Available:

<https://nodejs.org/en/docs/>

[20] Face-api.js, "Real-time face detection and recognition in the browser," [Online].

Available: <https://github.com/justadudewhohacks/face-api.js>

[21] OWASP Foundation, "Top 10 Web Application Security Risks," [Online]. Available:

<https://owasp.org/www-project-top-ten/>

[22] Mongoose ODM, "Mongoose Documentation," [Online]. Available: 86

<https://mongoosejs.com/docs/>

[23] Firebase, "Firebase Cloud Messaging," Google Developers, [Online]. Available:

<https://firebase.google.com/docs/cloud-messaging>

[24] Postman, "API Platform for Building and Using APIs," [Online]. Available:

<https://www.postman.com/>

[25] ISO/IEC 27001, Information Security Management Systems – Requirements.

International Organization for Standardization, 2013.

[26] R. Fielding and R. Taylor, "Architectural Styles and the Design of Network-based Software Architectures," University of California, Irvine, 2000.

[27] M. Fowler, Patterns of Enterprise Application Architecture. Addison-Wesley, 2002.

[28] D. Flanagan, JavaScript: The Definitive Guide, 7th ed., O'Reilly Media, 2020.

[29] B. Kernighan and D. Ritchie, The C Programming Language, 2nd ed., Prentice Hall, 1988.

[30] K. Beck, Test-Driven Development: By Example. Addison-Wesley, 2003.

[31] J. Resig and B. Bibeault, Secrets of the JavaScript Ninja, 2nd ed., Manning Publications, 2016.

[32] W. Stallings, Cryptography and Network Security, 7th ed., Pearson, 2016.

[33] C. Allen, The Ethereum Yellow Paper, Ethereum Foundation, [Online]. Available:

<https://ethereum.org/en/whitepaper/>

[34] GitHub Docs, "GitHub Actions Documentation," [Online]. Available:

<https://docs.github.com/en/actions>

[35] J. MacCormick, Nine Algorithms That Changed the Future: The Ingenious Ideas That Drive Today's Computers. Princeton University Press, 2011.